

Übungen Betriebssysteme

Aufgaben

Zeitplanung der Übungen	2
Anmerkungen	2
Übung 1	3
Aufgabe 1.1 Umgang mit der Shell	3
Aufgabe 1.1.1 (Shell-Grundlagen und einfache Kommandos)	3
Aufgabe 1.1.2 (Statuscode)	4
Aufgabe 1.1.3 (Filter und Pipelines)	4
Aufgabe 1.1.4 (Kommandosubstitution)	5
Aufgabe 1.1.5 (Einfaches Shellscripting)	5
Aufgabe 1.2 Verkettete Listen, Dynamischer Speicher, Ein und Ausgabe	7
Übung 2	10
Aufgabe 2.1 Kopieren von Dateien und System Calls	10
Aufgabe 2.1.1 Programm zum Kopieren von Dateien	10
Aufgabe 2.1.2 System Calls Trace (Nur bei Verwendung von Linux)	10
Aufgabe 2.2 Linux Kernel Module (LKM)	12
Aufgabe 2.2.1 Kompilieren und Laden eines LKM	12
Aufgabe 2.2.2 <code>printk</code>	14
Aufgabe 2.2.3 Kommunikation Userspace/Kernelspace mittels <code>proc</code> -Dateisystem	14
Ihre Aufgabe: Ausgabe der vergangenen Sekunden seit Modulladezeitpunkt	15
Übung 3	16
Aufgabe 3.1 Verständnisfragen	16
Aufgabe 3.2 Fork/Exec/Wait	17
Aufgabe 3.3 (Ein-/ Ausgabeumlenkung)	19
Aufgabe 3.4 Pipes	20

Zeitplanung der Übungen

Die Ausgaben der Übungen richten sich nach dem Fortschritt der Vorlesung aus. Zu Ihrer Orientierung finden Sie nachfolgend eine Empfehlung *bis wann* Sie die jeweiligen Übungen *spätestens* vollständig abgeschlossen haben sollten. *Beachten Sie: Es handelt sich hierbei nur um Empfehlungen. Die Übungen selbst haben keine festen Abgabetermine.*



Es wird Ihnen **dringend** angeraten, alle Übungen vollständig und gewissenhaft zu erledigen, da sie Sie auf die Hausübungen vorbereiten, bzw. schon Einzelteile der Hausübung lösen, die dann später nur noch angepasst werden müssen.

Übungsnummer und Themen	Ausgabe	Fertig bis
Übung 1: Bash und Zahlenliste	erste Vorlesungswoche	zweite Vorlesungswoche
Übung 2: Kopieren, System Calls und LKM	erste Vorlesungswoche	dritte Vorlesungswoche
Übung 3: Prozesse: Fork, Exec, Pipes	erste Vorlesungswoche	vierte Vorlesungswoche

Nach Übung 3 erfolgt die Ausgabe der Hausübung 1 in gesonderter Art.

Anmerkungen



- Für die praktischen Aufgaben der Übungen benötigen Sie ein Linux System.
 - Für manche Übungen reicht es nicht aus Mac OSX oder Windows WSL zu verwenden!
 - Die Verwendung einer virtuellen Maschine (VM), also z.B. mittels VirtualBox, VMWare oder Qemu, mit installiertem Linux System, ist dringend angeraten, auch wenn Sie als Host-System bereits Linux verwenden!

Die Dokumentation aller Kommandos, Systemaufrufe und Standard-Funktionen und -Programmen sind System-Manual verfügbar, das mit dem Kommando `man` aufgerufen wird. Machen Sie sich mit den Manpages und deren Verwendung vertraut (`man man`). Lesen Sie hier insbesondere den Abschnitt **DESCRIPTION**. Die Dokumentation der eingebauten SHELL-Befehle kann mit bei der BASH-Shell mit dem Kommando `help` gelesen werden. Die Manuals müssen ggf. als separate Pakete (`manpages`, `manpages-dev`, ...) installiert werden.

Machen Sie sich mit einem Texteditor ihrer Wahl vertraut. Einige Beispiele:

- GUI:
 - [Visual Studio Code](#), [Atom](#)
 - [CLION](#) (Lizenz: [Lizenz](#)), [Sublime Text](#), [Gedit](#)
- CLI: vim, nano, joe, emacs

Übung 1

Aufgabe 1.1 Umgang mit der Shell

Öffnen sie eine Terminal und bearbeiten Sie die folgenden Aufgaben.

Aufgabe 1.1.1 (Shell-Grundlagen und einfache Kommandos)

Als Shell bezeichnet man einen Kommandointerpreter. Z.B. **bash** (Bourne Again SHell). Eine Shell kann einfache Kommandos, aber auch komplexe Programme ausführen. Dabei spielt es keine Rolle, ob die Kommandos interaktiv eingegeben werden oder in einer Datei (Shellscript) gespeichert sind.

A. Lesen Sie ein Tutorial zur BASH:

Beispiele:

- <https://ubuntu.com/tutorials/command-line-for-beginners#1-overview>
- <https://linuxconfig.org/bash-scripting-tutorial>
- <https://ryanstutorials.net/bash-scripting-tutorial/>

Außerdem hier der Link zum bekannten Linux One Page Manual, was viele wichtige Kommandos auf zwei Seiten knapp darstellt:

<https://www.cheat-sheets.org/saved-copy/The%20One%20Page%20Linux%20Manual.pdf>

B. Machen Sie sich mit dem System-Manual vertraut:

- Lesen Sie den Eintrag zum Befehl **pwd** komplett (**man pwd**).
- Lesen Sie, was in der Beschreibung zum Befehl **man** **man** über die Einteilung des System-Manuals in verschiedene Sektionen steht.
 - Wie wählt man eine Sektion aus?

C. Lesen Sie die Kurzbeschreibung zum Befehl **apropos**

und probieren Sie den Befehl aus!

D. Lesen Sie im System-Manual die grundlegende Verwendung (ohne Details) folgender Befehle:

echo, pwd, ls, ps, mkdir, rmdir, cat, rm, cp, mv, head, tail, grep

E. Machen Sie sich mit der Shell durch interaktive Ausführung der nachfolgenden Befehle vertraut:

```
$ cd # gehe in das Heimatverzeichnis
$ V=bs/aufgaben/a1 # Pfad des neuen Verzeichnisses in Variable V merken
$ mkdir -p $V # erzeuge neues Verzeichnis
$ cd $V # gehe in das neue Verzeichnis
$ touch d # erzeuge leere Datei "d"
$ ls -l d # Dateiattribute von "d" anzeigen
$ ls -l # Dateiattribute von Dateien im Verzeichnis anzeigen
$ ls -la # Wo ist der Unterschied zum letzten Kommando?
```

- Welche Ausgaben erwarten Sie am Bildschirm in der oben beschriebenen Situation bei den unten angegebenen Kommandos?
 - Beachten Sie dabei: `ls -l d e` soll alle Attribute der Dateien `d` und `e` anzeigen, eine Datei `e` existiert aber nicht! TIP: Die spezielle Datei `/dev/null` ist bei der Ausgabe eine Art schwarzes Loch, in der alle Ausgabedaten verschwinden. Beim Lesen erhält man sofort `EOF` → End-Of-File.

```
$ ls -l e
$ ls -l d e
$ ls -l d e > /dev/null
$ ls -l d e 2> /dev/null
$ ls -l d e > /dev/null 2> /dev/null
$ ls -l d e > /dev/null 2>&1
```

Aufgabe 1.1.2 (Statuscode)

Ein Kommando liefert einen ganzzahligen Statuscode zurück: 0 heißt immer **OK**, andere Werte stehen für bestimmte Fehler. Den Statuscode des letzten Kommandos zeigt man an mit `echo $?` an. Betrachten Sie noch einmal das Beispiel von oben:

Welche Statuscodes liefern die nachfolgenden Kommandos?

```
$ ls -l e; echo $?
$ ls -l d; echo $?
$ ls -l d e; echo $?
```

Aufgabe 1.1.3 (Filter und Pipelines)

Ein Filter ist ein Programm, das einen Strom von Eingabedaten transformiert und die Ergebnisse der Transformation als Ausgabestrom weiterleitet. Filter kann man in einer Pipeline nach dem Fließbandprinzip hintereinander anordnen.

A. Verwenden Sie folgende Filter einzeln interaktiv und beobachten Sie die Wirkung.

```
cat
wc
tr e x
tr -d e
grep [0-9]
grep ^[0-9]
awk '{print $2}'
```

Eine geeignete Testeingabe ist beispielsweise:

- 1 Hallo Welt
- 2 Ende

- Gleichung: $12 + 24 = 36$

Bei interaktiver Eingabe erzeugen Sie mit der Tastenkombination **STRG-D** das Eingabeende (**EOF**).

B. Betrachten Sie die nachfolgenden Pipelines |.

Probieren Sie die Befehle einzeln aus. Welche Funktion haben die Pipelines?

```
$ ls -l | grep ^d
$ ls -l | grep ^d | wc -l
$ ls -l | grep ^d | awk '{print $9, $1}'
$ echo 4 + 3 | bc
$ echo "sqrt(2)" | bc -l
```

Aufgabe 1.1.4 (Kommandosubstitution)

A. Testen Sie Kommandosubstitution mit folgenden Beispielen:

```
$ date "+%Y-%m-%d"
$ DATE=`date "+%Y-%m-%d"`; echo $DATE
$ DATE=$(date "+%Y-%m-%d"); echo $DATE
$ date
$ TAG=$(date|awk '{print $1}'); echo $TAG
```

B. Zuweisung bei Rechenoperationen

Wie kann man an die Variable WURZEL2 die Quadratwurzel von 2 zuweisen?

Aufgabe 1.1.5 (Einfaches Shellscripting)

A. Ein neues Kommando **anzahlsubdirs** soll die Anzahl der Unterverzeichnisse eines beliebigen Verzeichnisses ausgeben.

Die folgende Alias-Definition zählt nur die Unterverzeichnisse des aktuellen Verzeichnisses. Erweitern Sie die Definition zu einer Funktion, die das Verzeichnis als Parameter erhält. Testen Sie die Funktion interaktiv und als Shellscript.

```
alias anzahlsubdirs="ls -l | grep ^d | wc -l"
```

B. Schreiben Sie ein Shellscript **showargs**, das seine Argumente wie im folgenden Beispiel ausgibt:

```
$ showargs hallo Welt
"hallo"
"Welt"
```



Ausführung entweder mit **bash showargs** oder verwenden sie den Befehl **chmod +x showargs** um Ihr Shellscript ausführbar zu machen.



Shellscripte in Dateien fangen normalerweise in Zeile 1 mit einem `#!/bin/bash` (*shebang*) an um den eigentlichen Kommandointerpreter auszuwählen. Siehe: <https://de.wikipedia.org/wiki/Shebang>

C. Erweitern Sie das Shellscript so, dass es die Argumentnummern mit ausgibt:

```
$ showargs hallo Welt  
1. "hallo"  
2. "Welt"
```



Die Shell führt normalerweise keine Berechnungen mit Variablen durch.

Beispiel:

```
$ i=1; i=$((i+1)); echo $i  
1+1
```

Man kann hier Hilfsprogramme wie `expr` oder `bc` und Pipelines verwenden. Einfacher geht es mit dem BASH-internen Kommando `let` (siehe `help let`): `let i=1; let i++`

Aufgabe 1.2 Verkettete Listen, Dynamischer Speicher, Ein und Ausgabe

Schreiben Sie ein Programm in der Programmiersprache C, das beliebig viele positive Zahlen via Standardeingabe erwartet und nach jeder eingegebenen Zeile eine sortierte Liste aller bisher eingegebenen Zahlen ausgibt. Zahlen werden mit Leerzeichen getrennt eingegeben. Das Programm muss unter Linux lauffähig sein! Bei dem drücken der *ENTER-Taste* → Ausgabe der Liste + Warten auf nächste Eingabe:

Beispiel

```
$/zahlenliste
Eingabe: 1 3 2
Ausgabe: 1 2 3
Eingabe: 7 6 5
Ausgabe: 1 2 3 5 6 7
Eingabe: 1 2
Ausgabe: 1 2 3 5 6 7
```

Für die Verwaltung der Liste verwenden Sie eine einfach oder doppelt verkettete Liste (*linked-list*). Verwenden Sie hierzu eine geeignete Struktur (**struct**), die sowohl die Zahlen aufnehmen kann und auch deren Verkettung erlaubt. **WICHTIG:** Verwenden Sie dynamischen Speicher (*Heap*) und **keine Arrays** für die Liste! Speicher auf dem Heap allokieren Sie mit **malloc()**!



Fügen Sie die eingegebenen Zahlen einfach der Liste hinzu und verwenden sie einen geeigneten Sortieralgorithmus vor der Ausgabe, z.B. *Bubblesort*. Ein Beispiel für einen *Bubblesort* in C finden Sie z.B. [hier](#).



Die meisten Algorithmen im Netz arbeiten mit einem simplen Array um eine Sortierung zu demonstrieren. Hierbei ist oftmals die Länge (Größe) des Arrays bekannt. Ihr Programm soll jedoch als eine verkettete Liste implementiert werden, hierbei ist es **nicht** nötig (und in der Aufgabenstellung auch nicht gewollt) eine vorgegebene Länge (Größe) der Liste zu festzulegen.

Ihr Programm sollte mit dem Aufruf von **gcc -std=c99 -Wall -pedantic -o zahlenliste zahlenliste.c** ohne die Ausgabe von Fehlern kompilieren.

Bei der Tastenkombination **STRG-C** wird ihr Programm per Signal **SIGINT** beendet.



Folgende Funktionen könnten hilfreich sein:

malloc printf free strchr strtok strlen strtol atoi

Sollten Ihnen die Funktionen unbekannt sein, lesen Sie die jeweilige Manpage!

Zusatz 1: Ignorieren von doppelt eingegebenen Zahlen

Implementieren Sie eine Funktion die doppelt eingegebene Zahlen ignoriert und nicht in die Liste einfügt.

Zusatz 2: Ignorieren von ungültigen Eingaben

Implementieren Sie eine Funktion die erkennt ob die Eingaben auch wirklich Zahlen sind. Alle Eingaben die nicht Zahlen entsprechen sollen ignoriert werden.

Zusatz 3: Speicherbereinigung

Implementieren Sie eine Funktion die den verwendeten Speicher wieder vollständig freigibt. Registrieren Sie diese Funktion als einen *signal-Handler*, der bei dem Signal **SIGINT** aufgerufen wird.



```
void cleanup_memory() {
    ...
}

int main (...){
    ...
    signal(SIGINT, cleanup_memory);
    ...
}
```

- Frage: Warum muss bei der Verwendung der Speicherbereinigung die Liste als globale Variable definiert sein?
- Frage: Wie müsste man das Programm verändern um die Funktion `cleanup_memory()` bei normalem beenden des Programms (d.h. z.B. bei `return 0` in `main()`) aufzurufen? Schauen Sie sich hierzu die Funktion `atexit` an.
- Frage: Wenn Sie in Ihrem Signal-Handler kein `exit(0);` Aufruf machen, und 2 mal **STRG+C** (**CTRL+C**) drücken, wieso wird Ihr Signal-Handler nur einmal aufgerufen? (Schauen Sie hierzu in die Manpage von signal `man 2 signal`)

Vollständiges Beispiel mit doppelten und ungültigen Zahlen sowie Speicherbereinigung

```
$ ./zahlenliste
Eingabe: 1 3 2
Ausgabe: 1 2 3
Eingabe: 1 4
Ausgabe: 1 2 3 4
Eingabe: 10 9 1 8
Ausgabe: 1 2 3 4 8 9 10
Eingabe: 7foo 7 bar7
Ausgabe: 1 2 3 4 7 8 9 10
Eingabe: foo bar
Ausgabe: 1 2 3 4 7 8 9 10
Eingabe: ^C
=====
Cleaning up memory
```


=====

Zusatz 4: Pointer-basierte Sortierung

Bei einem einfachen Bubblesort werden oftmals nur die Werte in der Listenstruktur getauscht.

```
/*
 * swap number values during bubblesort
 */

void swap(struct Number *a, struct Number *b){
    int temp=a->value;
    a->value=b->value;
    b->value=temp;
}
```

Nehmen Sie nun an, dass in der Struktur sehr viele Werte (10, 20, 100, 1000000) vorhanden sind. U.U. auch Daten wie Strings die nicht einfach mit einer einfachen Zuweisung (d.h. mittels `=`-Operator) zugewiesen oder kopiert werden können.

Überlegen Sie welche Nachteil diese Art der *einfachen* Implementierung der Funktion `swap` (wie oben gezeigt) hat, falls komplexere Daten in der Struktur vorhanden sind.

- Wie könnte man `swap` anpassen damit eine direkte Zuweisung der Daten (Werte) nicht mehr nötig ist?
- Wäre es von Vorteil hierfür eine doppelt verkettete Liste zu verwenden? Wenn ja, warum?
- Implementieren Sie ein Pointer-basierte Sortierung der Liste für Ihr Programm.

Übung 2

Aufgabe 2.1 Kopieren von Dateien und System Calls

Lesen Sie die Manpages zu den System Calls `open`, `read` und `write`. Beispiel für `read` → `man 2 read`.

Aufgabe 2.1.1 Programm zum Kopieren von Dateien

Schreiben Sie ein C-Programm das unter Linux lauffähig ist und eine Datei kopiert. Verwenden Sie dazu die Funktionen `open`, `read` und `write`.

Aufruf-Syntax: `copy <INPUT> <OUTPUT>`

- `<INPUT>` ist der Pfad der zu kopierenden Datei.
- `<OUTPUT>` ist der Pfad zu der Kopie.
- Wenn die Datei nicht vorhanden ist, soll sie neu erzeugt werden. Wenn sie schon vorhanden ist, soll der alte Inhalt gelöscht und durch den neuen Inhalt vollständig ersetzt werden.



Das eigentliche Kopieren der Dateiinhalte (mit `read` und `write`) kann mittels einer Schleife in 2 Zeilen vollständig implementiert werden.

Aufgabe 2.1.2 System Calls Trace (Nur bei Verwendung von Linux)

Sie benötigen das Programm `strace`. Falls das Programm nicht installiert ist können Sie es mittels `sudo apt-get install strace` unter Debian/Ubuntu installieren.

Rufen Sie Ihr erstelltes Programm mittels `strace copy <INPUT> <OUTPUT>` auf und beobachten Sie die Ausgabe.

Beantworten Sie folgende Fragen:

1. Wie viele System Calls ruft Ihr Programm insgesamt auf [Anzahl]?
2. Wie viele unterschiedliche System Calls ruft Ihr Programm auf [Anzahl]?
3. Wo werden die System Calls aufgerufen die Sie in Ihrem Programm verwendet haben?
 - a. Können Sie eindeutig identifizieren und bestimmen wo und wann Ihre System Calls aufgerufen werden?
4. Warum werden viel mehr (und z.T. auch andere) System Calls aufgerufen als Sie implementiert haben?

Experimentieren Sie mit `strace` und anderen *einfachen* Programmen. Ein gutes Beispiel hierfür wäre z.B. das Programm `cp`, das zu Ihrem Programm funktionell sehr ähnlich ist.

Beantworten Sie folgende Fragen:

1. Beobachten Sie die Ausgaben von `strace`. Verhalten sich die anderen Programme ähnlich zu Ihrem Programm?
2. Rufen diese Programme vielleicht sogar (teilweise) genau die exakt selben System Calls auf wie Ihr Programm?
3. Wie erklären Sie sich dieses Verhalten?

Aufgabe 2.2 Linux Kernel Module (LKM)

Für diese Aufgabe benötigen Sie die bereitgestellte Linux VM oder ein eigenes Linuxsystem auf Ihrem Rechner. Bei der Verwendung eines eigenen Systems, stellen Sie sicher, dass Sie die Header-Dateien Ihres Kernels installiert haben. Unter Debian heißt das zuständige Paket z.B. `linux-headers-amd64` was sie mittels `sudo apt-get install linux-headers-amd64` installieren können.



Für andere Distributionen, erkundigen Sie sich selbständig wie das Paket heißt und installieren Sie es.

Falls Sie das GIT-Repository zum Kurs noch nicht auf Ihrem System haben, holen Sie dies nach:

SSH

```
$ git clone git@git.thm.de:arin07/betriebssysteme_public.git
```



HTTPS

```
$ git clone https://git.thm.de/arin07/betriebssysteme_public.git
```

Falls Sie das Repository bereits geklont haben, wechseln in das entsprechende Verzeichnis `cd <ihr_pfad>` und aktualisieren Sie das Repository mittels `git pull`.

Aufgabe 2.2.1 Kompilieren und Laden eines LKM

Für diesen Teil der Übung, verwenden wir das in der Vorlesung vorgestellte LKM.

```
/*
 * hello-1.c - The simplest kernel module.
 */
#include <linux/module.h> /* Needed by all modules */
#include <linux/kernel.h> /* Needed for KERN_INFO */

int init_module(void)
{
    printk(KERN_INFO "Hello world!\n");

    /*
     * A non 0 return means init_module failed; module can't be loaded.
     */
    return 0;
}

void cleanup_module(void)
{
    printk(KERN_INFO "Goodbye world!\n");
}
```

```
}
```

```
MODULE_LICENSE("GPL");
```

Wechseln Sie in das Verzeichnis mit dem Sourcecode des LKM und bauen Sie es mit dem Aufruf von `make`. Der relative Pfad ist: `uebungen/uebung2/code/lkm/hello/`

Lesen Sie die Manpages zu `modinfo`, `insmod`, `lsmod`, `dmesg` und `rmmod` und probieren Sie die Befehle aus:

1. Schauen Sie sich die Ausgabe von `sudo modinfo hello_module.ko` an.
2. Laden Sie das LKM mittels `sudo insmod hello_module.ko`.
3. Prüfen Sie mittels `lsmod | grep hello_module` ob das LKM geladen wurde.
4. Entfernen Sie das LKM mit `sudo rmmod hello_module`.
5. Verwenden Sie den Befehl `sudo dmesg` und finden Sie die Ausgaben `Hello world!` und `Goodbye world!`.
 - a. Als Alternative zu `sudo dmesg` kann auf aktuellen Systemen auch der Befehl `sudo journalctl -k -b 0` verwendet werden.

Wenn alles funktioniert: Herzlichen Glückwunsch, Sie haben Ihr erstes LKM gebaut, geladen und entfernt!



U.U kann es passieren, dass Sie bei dem Aufruf von `sudo insmod <KERNEL MODUL>` eine Fehlermeldung bekommen die sagt, dass die Modulversion nicht zu Ihrem Kernel passt. In diesen Fällen führen Sie folgende Zeile im Terminal aus: `sudo apt-get install linux-headers-amd64 linux-image-amd64`. Anschließend starten Sie Ihr System neu und versuchen es erneut!

Aufgabe 2.2.2 `printk`

Eine der wichtigsten Funktionen für LKMs ist `printk`. Dieser dient zur einfachen Ausgabe aller Art und wird generell wie `printf` verwendet. Außerdem stellt `printk` eine sehr einfache Möglichkeit zum Debugging bereit. Die Ausgaben von `printk` landen im sog. `kernel ring buffer` den Sie sich mittels `sudo dmesg` anzeigen lassen können.

Im Fall unseres LKM, haben wir `printk` folgendermaßen verwendet:

- `printk(KERN_INFO "Hello world!\n");` und
- `printk(KERN_INFO "Goodbye world!\n");`

was entsprechende Ausgaben erzeugt hat. Das Makro `KERN_INFO` gibt hierbei das sog. *loglevel* an. Weitere *loglevel* und Informationen dazu finden Sie [hier \(html\)](#) oder [hier auf S.77 ff.](#)

Experimentieren Sie mit `printk` indem Sie:

1. den Sourcecode anpassen in `hello_module.c`.
2. das LKM bauen.
3. das LKM laden.
4. mittels `sudo dmesg` Ihre Ausgaben ansehen.
5. das LKM entfernen.

Was erzeugt der Aufruf folgender `printk` Funktion?

```
printk(KERN_INFO "Hello world from module [%s]->[%s]:[%i]!\n", __FILE__, __func__, __LINE__);
```

Aufgabe 2.2.3 Kommunikation Userspace/Kernelspace mittels `proc`-Dateisystem

Wechseln Sie in das Verzeichnis `uebungen/uebung2/code/lkm/proc/` und schauen Sie sich den Sourcecode des LKM `proc.c` an. Wenn Sie dieses LKM bauen und laden, wird eine Datei unter `/proc/hello` angelegt.

Wenn Sie nun lesend auf diese Datei zugreifen, z.B. `cat /proc/hello`, erfolgt eine Ausgabe auf Ihrem Terminal. Wie sie sehen, bedarf es einiges an Code damit eine Ausgabe auf dem Terminal erscheint. Grundsätzlich ist die Kommunikation zwischen Kernel und Userspace-Programmen umständlich, da hier zwei voneinander getrennte Speicherbereiche verwendet werden. Die Operation `copy_to_user` erledigt hierbei die Arbeit des Kopierens von Inhalten aus einem Puffer des Kernelspace in einen Puffer der im Userspace zur Verfügung steht. Das Pendant, das Daten vom Userspace zum Kernelspace kopiert, ist `copy_from_user`. Lesen Sie sich hierzu unter <https://ezs.kr.hsnr.de/TreiberBuch/html/sec.datentransfer.html> kurz ein.

Versuchen Sie das LKM vollständig zu verstehen und diskutieren Sie mit Ihren Kommilitonen dessen Funktion.

Ihre Aufgabe: Ausgabe der vergangenen Sekunden seit Modulladezeitpunkt

Ihre Aufgabe ist es nun ein LKM zu entwickeln, das die vergangenen Sekunden seit seiner Ladezeit mittels einer Datei in `/proc/seconds` ausgibt. Weiterhin soll eine Datei `/proc/jiffies` angelegt werden, die den aktuellen Wert dieses Zählers ausgibt, z.B. `Current Jiffies: 4333569968`.

Wenn Sie also `cat /proc/seconds` ausführen soll eine Ausgabe wie z.B. `Module loaded [5 s] ago!` erfolgen. Die Berechnung der Sekunden erfolgt anhand des sog. `jiffies`-Zählers im Kernel. Dieser Zähler wird pro Sekunde um einen bestimmten Wert inkrementiert und kann demnach in Sekunden umgewandelt werden. (`jiffies/HZ` ergibt einen Sekundenwert, wobei `HZ` eine Konstante ist.)

Wechseln Sie in das Verzeichnis `uebungen/uebung2/code/lkm/jiffies_time/` und gehen Sie folgendermaßen vor:

Ändern bzw. ergänzen Sie den Code in `jiffies_time.c` so, das er die o.a. Spezifikation erfüllt.

1. Anlegen von 2 Dateien (`/proc/jiffies` und `/proc/seconds`).
2. Bei Aufruf von `cat /proc/jiffies` → Ausgabe des aktuellen Jiffie-Werts.
3. Bei Aufruf von `cat /proc/seconds` → Ausgabe der vergangenen Sekunden seitdem das Modul geladen wurde.
4. Löschen der 2 Dateien (`/proc/jiffies` und `/proc/seconds`) bei `rmmod`.



- Wenn Sie Speicher auf dem Heap benötigen sollten, können Sie dies mit `kmalloc(size_t size, GFP_KERNEL)` tun.
 - <https://www.kernel.org/doc/html/docs/kernel-api/API-kmalloc.html>
- Geben Sie den Speicher auch bitte wieder frei, wenn er nicht mehr benötigt wird. Das geschieht mit `kfree(void * ptr)`.
 - <https://www.kernel.org/doc/html/docs/kernel-api/API-kfree.html>

Funktionen die Sie möglicherweise benötigen: `kmalloc`, `kfree`, `strcmp`, `sprintf`, `printk`, `copy_to_user`

Übung 3

Aufgabe 3.1 Verständnisfragen

- Schätzen Sie ohne vorher nachzusehen:
 - Wie viele Argumente kann man maximal einem Programm übergeben?
 - Wie groß darf ein einzelnes Argument sein?
- Überprüfen Sie ihre Schätzung durch Nachlesen im Systemmanual zum Systemaufruf `execve`
- Welche Gründe könnten dazu führen, dass ein `exec`-Aufruf fehlschlägt? Überlegen Sie zuerst und lesen Sie dann nach, ob Sie an alle möglicherweise auftretenden Probleme gedacht haben
- Bei bestimmten Ereignissen, z.B. wenn der Benutzer die Taste `Stgr-C` drückt, erhalten Programme Signale. In einem C-Programm kann eine Funktion als Signal-Handler für ein Signal genutzt werden. Der Funktionsaufruf erfolgt asynchron beim Empfang eines Signals
 - Einige Prozessattribute werden durch `exec` geändert, andere nicht. Warum werden die Signal-Handler und Exit-Handler beim `exec` zurückgesetzt?
- Was passiert beim `exec` mit den offenen Dateien?
- Ermitteln Sie durch Experimente, welchen Wert die Variable `errno` hat, wenn `exec` wegen fehlender Zugriffsrechte scheitert. Wie heißt der symbolische Name dieses Fehlers? Wie ist die Klartext-Fehlermeldung?



Lesen Sie unbedingt die Manpage von `execve` (`man 2 execve`). Dort finden Sie alle Informationen die Sie benötigen!

Aufgabe 3.2 Fork/Exec/Wait

Lesen Sie im System-Manual ([man](#)) die Einträge zu den Systemaufrufen `fork`, `exec` und `wait` und schauen Sie sich die besprochenen Folien aus der Vorlesung zu den jeweiligen Funktionen an. Sie dürfen gerne den Code aus der Vorlesung als Startpunkt verwenden bzw. einbinden.

Schreiben Sie ein Programm `factiwait`, dass zwei Subprozesse erzeugt (`fork()`), in denen jeweils ein Programm aufgerufen wird. Insgesamt entstehen also 3 Prozesse:

- Der Eltern Prozess
- 1. Kindprozess der `PROGRAMM 1` ausführt
- 2. Kindprozess der `PROGRAMM 2` ausführt



Überlegen Sie selbstständig welche `exec`-Variante Sie verwenden sollten!

Nach dem Start der beiden Subprozesse, wartet der Elternprozess (`wait()` oder `waitpid()`) bis beide Subprozesse beendet wurden und beendet sich dann selbst.

Die Aufrufsyntax des Programms soll folgendermaßen aussehen:

`activate <PROGRAMM 1> [<PARAMETER> . . .] <TRENnzeichen> <PROGRAMM 2> [<PARAMETER> . . .]`

Beispielaufruf bei gewähltem Trennzeichen `;;`:

```
$activate ls -l -a /tmp \; sleep 5
$activate ls -l -a . \; date
```



Falls ein `;` als Trennzeichen verwendet wird, muss dieses zuvor mit einem Backslash escaped werden → `\;`



Die Parameterlisten `[<PARAMETER> . . .]` von Programm 1 und Programm 2 können beliebig lang sein!

Sie können nachfolgenden Code als Basis für diese Aufgabe verwenden. Als Datei unter [uebungen/uebung3/code/fork_exec_pipe/factivate.c](#) zu finden.

factivate.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>
#include <string.h>
/*
 * These are all headers you need!
 */
```

```

int main(int argc, char *argv[]) {
    /*
     * Do as you were told.
     * Split argv[1..n] by using a <DELIMITER> into:
     * <PROGRAM 1> [<PARAMETER>...]
     * <PROGRAM 2> [<PARAMETER>...]
     */

    /*
     * create first child (fork()) and execute
     * <PROGRAM 1> [<PARAMETER>...] with exec*()
     */

    /*
     * create second child (fork()) and execute
     * <PROGRAM 1> [<PARAMETER>...] with exec*()
     */

    /*
     * Typically, here is the parent. Make sure this code is called
     * _after_ both subprocesses were launched!
     * Use wait() or waitpid() and wait until _both_ subprocesses are finished!
     * Then use exit()
     */
}

```



Achten Sie darauf ein Leerzeichen **vor** und **nach** Ihrem Trennzeichen zu verwenden →

<LEERZEICHEN><TRENNZEICHEN><LEERZEICHEN>

Dann können Sie einfach feststellen wann das Trennzeichen in `argv[]` vorkommt. Wenn Sie kein Leerzeichen verwenden, wird es komplizierter. Eine Lösung hierfür ist *optional* und *nicht nötig*.

Aufgabe 3.3 (Ein-/ Ausgabeumlenkung)

Lesen Sie im System-Manual den Eintrag zum Systemaufruf `dup2`. Schreiben Sie ein C-Programm `redirect`, das ein anderes Programm aufruft (`execlp` oder `execvp`) und dessen Standardausgabe `STDOUT_FILENO` und Standard-Fehlerausgabe `STDERR_FILENO` mit `dup2` in Dateien umgelenkt werden.

Die Aufrufsyntax des Programms soll folgendermaßen aussehen:

```
redirect <OUTPUT> <ERROR> <PROGRAM> [<PARAM> . . .]
```

- `<PROGRAM>` ist das aufzurufende Programm
- `<OUTPUT>` und `<ERROR>` sind die Pfade der Dateien, auf den die Ausgabe bzw. Fehlerausgabe umgelenkt werden sollen.
 - Wenn bei einem dieser Argumente statt eines Dateipfads ein Minuszeichen steht, soll die entsprechend Umlenkung nicht gemacht werden
- Hinter `<PROGRAM>` können beliebig viele Argumente stehen, die an das aufzurufende Programm als Parameter durchgereicht werden sollen.

Beispielaufrufe:

```
$redirect /tmp/test1.out - ps -aux  
$redirect /tmp/test2.out /tmp/test2.err cp /etc/hosts /etc/foo /tmp
```



Dieses Programm benötigt keine Subprozesse `fork()`. Sie können `exec*()` also direkt verwenden.

Aufgabe 3.4 Pipes

Lesen Sie im System-Manual die Einträge zu dem Systemaufruf `pipe` und schauen Sie sich die besprochenen Folien aus der Vorlesung zu Pipes noch einmal genau an. Sie dürfen gerne den Code aus der Vorlesung als Startpunkt verwenden bzw. einbinden. Schreiben ein C-Programm `pipeline`, dass eine Pipeline mit drei Subprozessen realisiert. Sie können dazu Ihr Programm `factiwait` aus [Aufgabe 3.2 Fork/Exec/Wait](#) erweitern.

Die Aufrufsyntax des Programms soll folgendermaßen aussehen:

```
pipeline <PROGRAM> [<PARAM>. . .] \| <PROGRAM> [<PARAM>. . .] \| <PROGRAM> [<PARAM>. . .]
```



Das Pipe Symbol muss hier escaped werden `\|`, da ansonsten die Shell selbst das Symbol interpretieren würde! Bei Symbolen die keine semantische Bedeutung in der jeweiligen Shell haben, muss das Symbol nicht escaped werden.

Der Elternprozess soll warten, bis alle Subprozesse fertig sind und dann selbst terminieren.



Implementieren und testen Sie Ihr Programm **in jedem Fall** mit mindestens 2 Pipes, so wie in den folgenden Beispielen gezeigt. Wenn Ihr Programm nur eine Pipe unterstützt, dann fehlt die Implementierung für die *Zwischenaufrufe*. Diese benötigen Sie aber später **zwingend** in der Hausübung!

Beispielausgabe:

```
$pipeline ps -aux \| cat \| grep bash
ar      8  0.0  0.0  18340  5456 pts/0    Ss   Nov01   0:02 -bash
ar     1259  0.0  0.0  14444  1108 pts/0    S    00:46   0:00 grep --color=auto
bash

$pipeline ls -l /tmp/ \| cat \| wc -l -c -m
87    5244    5244
```